FIRST PROJECT REVIEW · STUDY & VIVA GUIDE

Multi-Agent Explainable AI with Collaborative Neural Reasoning

Review 1 covers one thing: **building and training our own ~1 million parameter decoder-only Transformer language model from scratch.** No pretrained weights, no fine-tuning, no API model. This guide teaches exactly what you need to present and defend that — and stops there.

1,331,968	8.3327	2.1407	8.51
PARAMETERS	STEP-0 LOSS = LN(4096)	VALIDATION LOSS	PERPLEXITY

How this guide is labelled — read this first

LEVEL 1	**Must know.** If you know only this, you can still pass the review. About half the guide.
LEVEL 2	**Should know.** What the panel asks when they push one step deeper.
LEVEL 3	**Optional.** Useful for understanding. Do not memorise under time pressure.
MEASURED	A number produced by a run that actually happened, traceable to a log file.
PROPOSED	A design choice or configuration. Not a result.

No number in this guide is invented. Every MEASURED value comes from run `run_a_001` on a Tesla T4, seed 1337. If you are asked something this guide does not answer, say "*we haven't measured that yet*" — that answer is always safe and always correct.

Contents

PART 1 · LEVEL 1

The project in ten minutes

If you understand this page, you can answer 60% of what the panel will ask.

What is the project?

Most AI systems answer a question by running **one** reasoning process and printing the result. That process checks nothing, offers no second opinion, and gives the user no basis for deciding whether to trust the answer. When it is wrong, it is wrong *confidently and silently*.

Our project builds a system where **several specialised agents** work on the same question — one solves it, one criticises the solution, one verifies it independently, one produces an alternative — and a **coordinator** combines their views into a final answer, together with a record of who agreed, who disagreed, and how confident each was.

What problem are we solving?

PROBLEM WITH A SINGLE REASONING PROCESS	WHAT OUR SYSTEM ADDS
One attempt, never challenged	An independent second attempt (Alternative Reasoner)
No self-checking	A Critic that looks for errors, and a Verifier that checks independently
Confidence carries no information	Confidence measured from the model's own probabilities, then calibrated
No account of *why*	A decision record: agents, conclusions, agreement, verification, uncertainty

What are we doing in Review 1 — and what are we NOT doing?

Review 1 IS this

- Choose and clean a dataset
- Build **our own** BPE tokenizer
- Implement **our own** decoder-only Transformer
- Initialise it with **random** weights
- Train it from scratch
- Validate, save a checkpoint, reload it
- Generate text and measure loss + perplexity

Review 1 is NOT this

- Not the multi-agent system
- Not reasoning training
- Not the explainability engine
- Not the web application
- Not fine-tuning GPT-2, Llama, Qwen or anything else
- Not a model that can answer questions

Say this out loud in the review. Naming the boundary makes you look in control. Being asked about it makes you look unprepared.

The one-sentence answer to "what have you built?"

"*We implemented a 1,331,968-parameter decoder-only Transformer language model ourselves, initialised it with random weights, and trained it from scratch on ~50 million tokens of TinyStories using our own byte-pair-encoding tokenizer. It reached a validation loss of 2.1407, perplexity 8.51, and it generates coherent English sentences. No pretrained weights were used at any stage.*"

Where Review 1 sits in the whole project

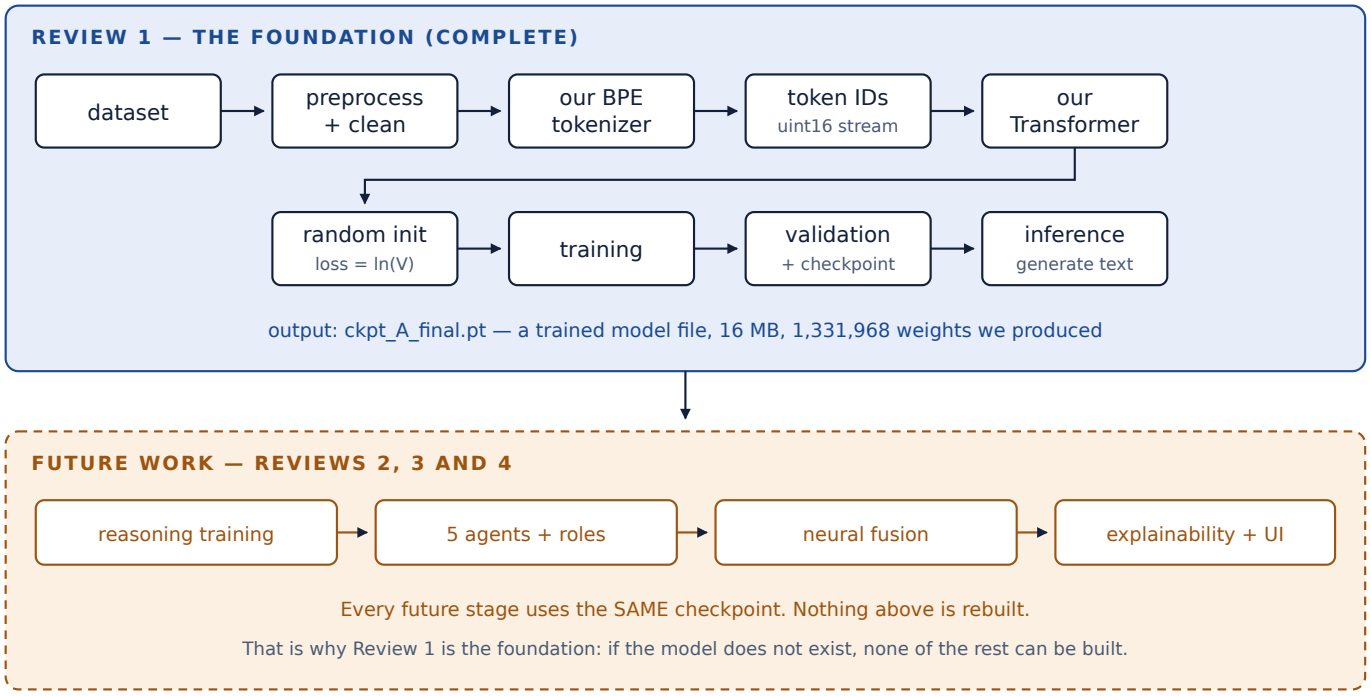
```
REVIEW 1 → Build + train our own LLM from scratch <-- YOU ARE HERE REVIEW 2 → Teach it to reason (structured reasoning training) REVIEW 3 → Multi-agent architecture: Solver, Critic, Verifier, Alternative, Coordinator REVIEW 4 → Collaborative neural reasoning + explainability + full evaluation
```

Memorise that four-line block — it is a complete answer to "*what comes after Review 1?*"

PART 1 · CONTINUED

Project at a glance

One diagram for the whole project. The shaded band is all of Review 1.



Reading the diagram. Boxes are stages; arrows mean "the output of this becomes the input of that". The blue band is what exists today. The dashed amber band is planned work — mention it in one sentence and move on.

The three sentences that carry the whole review

1. **What:** "We built and trained our own 1.33 million parameter language model from scratch."
2. **Proof:** "At step 0 its loss was 8.3327, which is exactly ln(4096) — the score of a model that knows nothing. A downloaded model would start far lower."
3. **Result:** "After 6,100 steps on 50 million tokens it reached validation loss 2.1407 and writes coherent English."

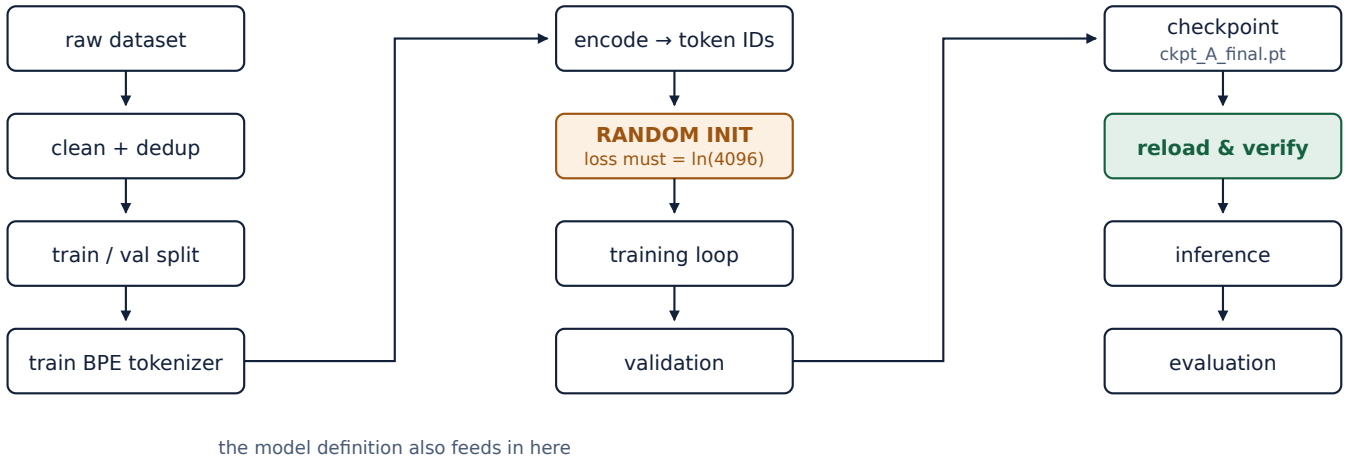
PART 2 · LEVEL 1

Review 1 scope — the pipeline, stage by stage

This is the single most important page in the guide. If the panel asks "explain what you did", you walk down this list.

#	STAGE	WHAT HAPPENS, IN ONE SENTENCE	OUR RESULT
1	**Dataset**	Choose a corpus of text simple enough that a tiny model can learn from it.	`TinyStories, 230,000 stories`
2	**Preprocessing**	Clean the text, remove duplicates, then split into train / validation / test.	`226,654 kept · 98/1/1`
3	**Tokenizer training**	Learn a vocabulary of 4,096 sub-word pieces from the training split only.	`3,966 merges learned`
4	**Encoding**	Convert every document into a sequence of integer token IDs.	`51,601,430 train tokens`
5	**Model definition**	Write the Transformer in PyTorch: embeddings, attention, feed-forward, LM head.	`1,331,968 parameters`
6	**Random initialisation**	Fill every weight with a random number drawn from N(0, 0.02). Nothing is loaded.	`step-0 loss 8.3327`
7	**Training**	Repeatedly predict the next token, measure the error, and adjust the weights.	`6,100 steps`
8	**Validation**	Measure loss on text the model never trained on, to check it is really learning.	`val loss 2.1407`
9	**Checkpoint**	Save the trained weights (plus optimiser state) to a file.	`ckpt_A_final.pt, 16 MB`
10	**Reload**	Load that file in a brand-new process and confirm it reproduces the same score.	`difference 0.000000`
11	**Inference**	Feed a prompt, sample tokens one at a time, decode back to text.	`15 samples generated`
12	**Evaluation**	Report loss, perplexity, next-token accuracy and generation quality.	`perplexity 8.51`

The same thing as a diagram



Read down each column, then across. Twelve stages, three columns.

Arrows mean data flowing forward. The amber box is the moment the model is created with random weights; the green box is the portability check that proves the checkpoint is a real, independent artefact.

If you are asked to summarise in 30 seconds

"Text goes in. We clean it, learn a 4,096-word sub-word vocabulary from it, and turn it into 51 million integers. We build a Transformer, fill it with random numbers, and train it to predict the next integer. When it gets good enough, we save the weights, reload them somewhere else to prove they're real, and generate text."

PART 3 · LEVEL 1

Dataset

Which dataset, and why that one

We use **TinyStories** — a collection of very short children's stories written with a deliberately small vocabulary, of the kind a three- or four-year-old would understand.

The reason this choice is defensible — know this cold

A model with only 1.33 million parameters cannot learn the full complexity of English. **Eldan & Li (2023)** showed that if you restrict the vocabulary and sentence structure — which is exactly what TinyStories does — models in the 1–30 million parameter range *can* produce fluent, coherent English. Wikipedia or news text at our scale produces gibberish. We did not pick TinyStories because it was easy; we picked it because it is the one corpus at which a model our size is known to work.

PROPERTY	VALUE
Name / source	TinyStories (`roneneldan/TinyStories` on Hugging Face)
License	**CDLA-Sharing-1.0** — open, permits research use with attribution
Content	Synthetically generated short stories, simple vocabulary, 2–5 sentences each
Full size available	2,141,709 stories (we did not need all of them)
Documents we loaded MEASURED	`230,000`
After cleaning + dedup MEASURED	`226,654 (1.5% dropped)`
Characters MEASURED	`204,352,155`
Split	98% train / 1% validation / 1% test, seeded shuffle (seed 1337)

How we preprocess it

```
download → strip control characters → Unicode NFKC normalise → length filter → remove duplicates → shuffle (seeded) → split 98/1/1 → fit tokenizer on TRAIN only → encode → pack into fixed-length blocks → save as uint16 file
```

Why each step is there LEVEL 2

STEP	REASON
Unicode NFKC normalise	Collapses look-alike characters so "fi" and "fi" are the same token, not two.
Length filter (40–8,000 chars)	Very short fragments teach nothing; very long outliers distort batching.
Deduplicate BEFORE splitting	**Important.** If a duplicate story lands in both train and validation, the model has already seen the "unseen" data and the validation score is falsely good.
Fit tokenizer on train only	Fitting on the whole corpus would leak validation statistics into the vocabulary. A small effect, free to avoid — and a panel may well ask.
Pack, do not pad	We join documents with an end-of-text token and slice into fixed 128-token blocks. Padding a 128-token window would waste a large fraction of a small compute budget.
Store as `uint16`	Our vocabulary is 4,096, which fits in 16 bits. Halves the file size and lets the training loop memory-map it instead of loading it into RAM.

From text to training data

```
"Once upon a time, there was a little girl." ↓ tokenizer [1, 3361, 432, 97, 507, 34, 341, 331, 97, 884, 1204, 36, 2] ↓ concatenate all documents, separated by <|eos|> [ ... 51,601,430 integers ... ] ↓ slice into windows of 129 tokens input = window[0:128] target = window[1:129] <-- shifted by ONE
```

The shift-by-one is the entire training objective — you will be asked about it

The target is the input moved one place to the left. So at every position the model sees tokens 1…t and must predict token t+1. One 128-token window therefore gives us **128 separate prediction problems**, not one. That is why so few documents produce so much training signal.

SPLIT	TOKENS MEASURED	USED FOR
train	51,601,430	updating the weights
validation	527,038	checking for overfitting during training
test	522,122	held back, untouched, for later stages

PART 4 · LEVEL 1

Tokenization and BPE

A neural network cannot read letters. It multiplies numbers. Tokenization is how text becomes numbers.

The five words you must be able to define

TERM	DEFINITION YOU CAN SAY OUT LOUD
Token	A chunk of text the model treats as one unit. Usually a whole word, a word-piece, or a punctuation mark — not always a full word.
Token ID	The integer that stands for that token. `"the"` might be ID 341. The model only ever sees the integers.
Vocabulary	The complete list of tokens the model knows. Ours has **4,096** entries, numbered 0 to 4,095.
Sequence	An ordered list of token IDs — one training example. Ours are 128 tokens long.
Special token	A reserved token that carries structure rather than meaning: begin-of-text, end-of-text, padding, unknown.

Why tokenization is necessary at all LEVEL 1

There are three obvious ways to turn text into numbers, and two of them fail:

APPROACH	PROBLEM
One ID per **character**	Vocabulary is tiny (good) but sequences become enormous. "understanding" is 13 steps instead of 1 or 2, so the model wastes its limited context length.
One ID per **word**	English has hundreds of thousands of words. The embedding table alone would dwarf our whole model — and any word not in the list becomes `<|unk|>`.
Sub-word (BPE) ✓	The compromise. Common words get one token; rare words are built from pieces. Small vocabulary *and* short sequences, and nothing is truly unknown.

What BPE actually does

Byte-Pair Encoding (Sennrich et al., 2016) is a compression idea applied to text. Start with every text as single characters. Then repeat: *find the most frequent adjacent pair of symbols in the corpus, and merge it into one new symbol.* Record each merge in order. Stop when the vocabulary reaches the size you wanted.

A worked example — small enough to reproduce on a whiteboard

Suppose the training corpus is just these words with these counts:

```
low     5      lower   2      newest  6      widest  3
```

Start as characters: `l o w · l o w e r · n e w e s t · w i d e s t`

MERGE	MOST FREQUENT PAIR	COUNT	NEW SYMBOL
1	`e + s`	6 + 3 = 9	`es`
2	`es + t`	9	`est`
3	`l + o`	5 + 2 = 7	`lo`
4	`lo + w`	7	`low`

After four merges, `low` is a single token instead of three, and `newest` is `n e w est` instead of six. The merge list *is* the tokenizer — encoding new text just means replaying those merges in the same order.

Our tokenizer MEASURED

PROPERTY	VALUE	WHAT IT MEANS
Vocabulary size	`4,096`	32 special + 98 characters + 3,966 learned merges
Merges learned	`3,966`	Each one is a rule like "e + s → es"
Character alphabet	`98`	Every character appearing ≥ 5 times in training
Round-trip accuracy	`1000 / 1000 (100%)`	encode → decode returns the original text exactly
`<|unk|>` rate	`0.0000%`	No character in validation fell outside our alphabet
Compression	`3.88 chars/token`	On average one token covers ~4 characters
Fingerprint	`d6080bac0a9a`	A hash of the vocabulary, stored inside every checkpoint

Why fit the tokenizer on the TRAIN split only?

Because the vocabulary is learned *from data*. If we learned it from the whole corpus, the merge list would encode statistics of the validation text, and our validation score would be slightly optimistic. It is a small effect and free to avoid — which is exactly the kind of thing a careful reviewer probes.

Special tokens, and why we reserved them before training

```
0  <|pad|>      4  <|solver|>        10  <|steps|>
1  <|bos|>      5  <|critic|>        11  <|answer|>
2  <|eos|>      6  <|verifier|>      12  <|conf|>
3  <|unk|>      7  <|alternative|>   13  <|issue|>
                8  <|coordinator|>   14  <|verdict|>
                9  <|question|>      15  <|evidence|>
                                   16-31  reserved for later
```

IDs 0–3 are the standard four: padding, begin-of-text, end-of-text, unknown. IDs 4–15 are for the *future* multi-agent stage.

Anticipate this question: "why are agent tokens in a Review 1 tokenizer?" Because adding a token later means inserting a brand-new randomly-initialised row into an already-trained embedding table — it trains badly and it invalidates every checkpoint made before it. Reserving 32 slots now costs under 0.2% of the model and means the tokenizer never has to be rebuilt. That is forward planning, not scope creep.

Text → token IDs, concretely

"Once upon a time, there was a little robot named Max." ↓ pre-tokenize (split on a regex, keep spaces visible) ["Once", " upon", " a", " time", ",", " there", " was", " a", " little", ...] ↓ apply the 3,966 merge rules in learned order ↓ look each piece up in the vocabulary [3361, 432, 97, 507, 34, 341, 331, 97, 884, 2813, 133, 120, 226, 154, 300, 67, 90, 36] ↓ decode (reverse the lookup, join, restore spaces) "Once upon a time, there was a little robot named Max." ← identical

PART 5 · LEVEL 1

What a language model is

The definition

A language model is a function that, given a sequence of tokens, outputs a probability distribution over what the next token will be.

That is the whole thing. It does not "understand" or "think". It assigns a probability to every one of our 4,096 tokens, given what came before.

Example

```
input:   "The cat sat on the"

output:  P(mat)   = 0.31
         P(floor) = 0.12
         P(chair) = 0.09
         P(table) = 0.06
         ... 4,092 more tokens, all with some probability ...
         P(banana)= 0.00002

         (all 4,096 numbers add up to exactly 1.0)
```

"LLM" — what makes it *large*? LEVEL 2

Nothing formal. "Large Language Model" describes Transformer language models with many parameters trained on a lot of text. GPT-3 has 175 billion parameters; ours has 1.33 million — roughly **130,000 times smaller**.

Say this before the panel says it. "Ours is a small language model. It shares the architecture of a large one — same decoder-only Transformer, same training objective — but at a scale we could genuinely build and train ourselves in a few weeks on a free GPU. Calling it an LLM would be overclaiming."

Next-token prediction

Training uses one task only: **given tokens 1…t, predict token t+1.** No labels, no human annotation — the text is its own answer key. This is called *self-supervised* learning.

```
Sentence:  "I love machine learning"

  I                       → love
  I love                  → machine
  I love machine          → learning
```

Three training examples from four words. A 128-token window gives 128 examples.

Autoregressive

Auto = self, **regressive** = feeding back. The model generates one token, appends it to its own input, and predicts the next from the extended sequence.

```
"Once" → model → "upon" "Once upon" → model → "a" "Once upon a" → model → "time" "Once upon a time" →
model → ...
```

The model never sees the future. It only ever extends what already exists — which is exactly what causal masking (Part 6) enforces during training.

What does the model actually learn?

It learns **statistical structure**, at several levels at once:

- **Spelling and morphology** — after `lit`, `tle` is likely.
- **Grammar** — after "The girl", a verb is more likely than another noun.
- **Local coherence** — if the story introduced "Lily", later pronouns should be "she".
- **Weak world knowledge** — cats sit on mats more often than on ceilings.

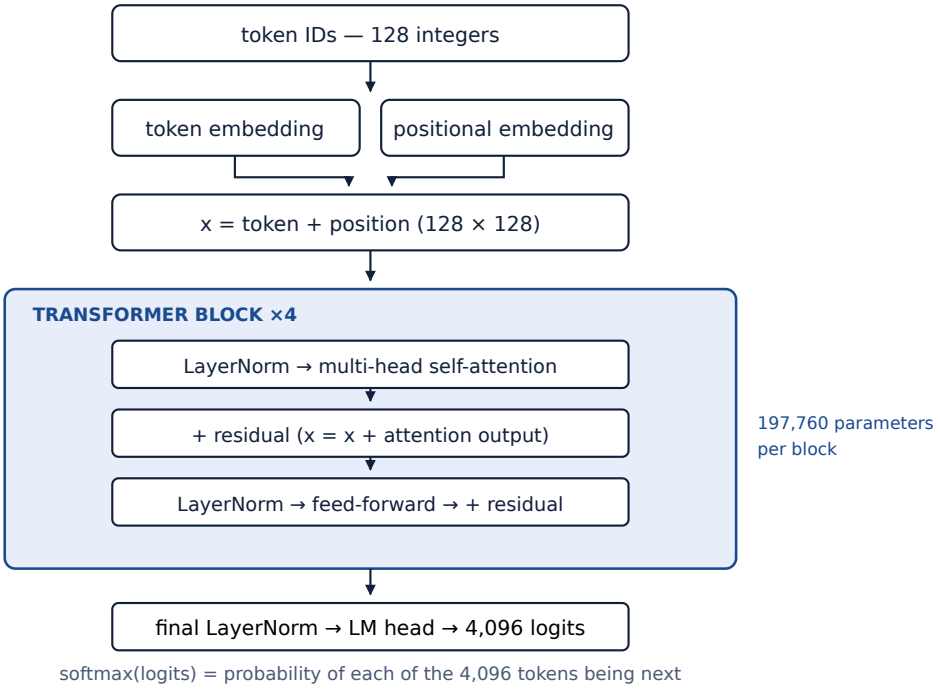
All of it is encoded in the 1,331,968 numbers. There are no rules, no grammar table, no dictionary — only weights that were adjusted until the predictions got better.

PART 6 · LEVEL 1 LEVEL 2

The Transformer

Each concept below gets four lines: what, why, how, and where it is in our model. Learn the "what" and "where" for all of them; learn "why" for the starred ones.

Our architecture, top to bottom



Every box is code we wrote. Data flows top to bottom. The blue region repeats four times — that is what "4 layers" means. The output is 4,096 numbers per position.

Concept by concept

★ Decoder-only architecture

What	A Transformer with only the decoder half — no encoder, no cross-attention.
Why	Encoder-decoder suits translation, where you read a whole input then write an output. We only ever *continue* text, so we need one stack that reads left-to-right. GPT-style models are all decoder-only.
Where	The whole of `model/transformer.py`.

★ Token embedding

What	A lookup table with one learned 128-number vector per vocabulary entry.
Why	Token ID 341 is just a label — 341 is not "more" than 340. The embedding replaces the arbitrary integer with a vector the model can do arithmetic on, and similar words end up with similar vectors.
How	A 4,096 × 128 matrix. Row *i* is the vector for token *i*.
Where	`tok_emb` — **524,288 parameters**, our largest single component.

★ Positional embedding

What	A second lookup table, one 128-number vector per *position* 0…127, added to the token embedding.
Why	**Attention has no built-in sense of order.** Without this, "dog bites man" and "man bites dog" look identical to the model. This is the most likely "why" question on this page.
Where	`pos_emb` — 128 × 128 = **16,384 parameters**. We use *learned absolute* positions: the simplest correct choice.

Residual connections

What	`x = x + f(x)` instead of `x = f(x)`.
Why	Gradients can flow straight back through the "+" without passing through `f`. Deep networks without residuals train badly or not at all. It also lets a block learn a small *adjustment* rather than a whole new representation.
Where	Twice in every block — after attention and after the feed-forward.

Layer normalisation

What	Rescales a vector so its values have mean 0 and variance 1, then applies a learned gain and bias.
Why	Keeps activations in a sane numerical range so training stays stable.
How	We use **pre-LN**: normalise *before* the sub-layer, not after. Post-LN needs a carefully tuned warmup or it diverges; pre-LN trains reliably out of the box.
Where	2 per block (8 total) + 1 final. 256 parameters altogether.

Feed-forward network (FFN)

What	Two linear layers with a GELU non-linearity between: 128 → 512 → 128.
Why	Attention *mixes information between positions*; the FFN *processes each position independently*. Both are needed. Widening to 512 and back gives the model room to compute something non-linear.
Where	Once per block — **131,712 parameters** each, the biggest part of a block.

LM head, logits and softmax LEVEL 1

What	A final linear layer mapping the 128-dim vector at each position to **4,096 numbers** — one score per vocabulary token. Those raw scores are the **logits**.
Why softmax	Logits are unbounded and can be negative. Softmax turns them into positive numbers that sum to 1 — a probability distribution.
How	$\text{softmax}(z)_i = \exp(z_i) / \Sigma_j \exp(z_j)$
Where	`lm_head` — and it is **weight-tied** to the token embedding: the same 4,096 × 128 matrix, used transposed. That saves 524,288 parameters, 39% of our total.

Weight tying — a good thing to volunteer

The embedding maps *token → vector*; the LM head maps *vector → token score*. They are inverse operations, so sharing one matrix is principled, not just a saving. Without tying our model would be 1,856,256 parameters; with it, 1,331,968.

PART 7 · LEVEL 2

Self-attention

The one piece of mathematics you should be able to write on a whiteboard.

The intuition first

Consider: "*The girl put the book on the table because **it** was heavy.*" To predict what follows, the model must work out that "it" refers to the book. Self-attention is the mechanism that lets the representation at the word "it" **pull in information** from the word "book" earlier in the sentence.

Every position asks: "*which earlier positions are relevant to me?*" — then builds a weighted blend of their information.

Q, K and V — the library analogy

NAME	STANDS FOR	ANALOGY
Q — query	what I am looking for	The search term you type into a library catalogue.
K — key	what I can offer	The label on each book's spine — used for matching.
V — value	what I actually contain	The contents of the book, which you take away once you've matched.

All three are produced from the *same* input by three different learned linear projections. That is why it is called **self**-attention: query, key and value all come from one sequence.

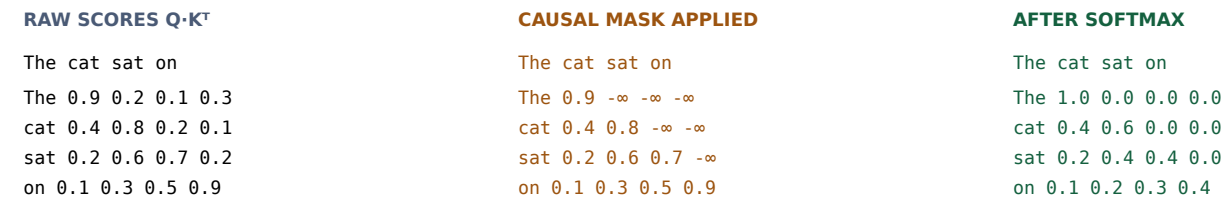
The equation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q K^{T} / \sqrt{d_k} \right) V$$

PIECE	WHAT IT DOES
`Q Kᵀ`	Dot product of every query with every key → a score for each pair of positions. High score = "this position is relevant to me".
`/ √dₖ`	**Scaling.** With $d_k = 32$ per head, dot products grow large; large values push softmax into a near-one-hot output where gradients vanish. Dividing by √32 keeps them in a workable range.
`softmax(...)`	Turns the row of scores into weights that are positive and sum to 1 — "how much attention to pay to each position".
`... V`	Weighted average of the value vectors. The output is a blend of the information the model decided was relevant.

Causal masking LEVEL 1

During training the model sees the whole sequence at once. Position 5 must not be allowed to look at position 6, or it would be predicting a token it can already see — and every loss number would be meaningless. So **before** the softmax we set all future scores to $-\infty$, which makes their softmax weight exactly zero.



Each row is one position asking "how much do I attend to each other position?"
The upper triangle is the future. $-\infty$ before softmax becomes exactly 0 after it.

Every row of the right-hand matrix sums to 1.0 — that is what softmax guarantees.

Numbers here are illustrative, chosen to show the shape of the mechanism — they are not from our model.

Multi-head attention

Instead of one attention over all 128 dimensions, we split into **4 heads of 32 dimensions each**, run attention independently in each, then concatenate the results.

Why: different heads can specialise. One might track subject–verb agreement, another local word order, another long-range references. One big attention would have to average all of those jobs together.

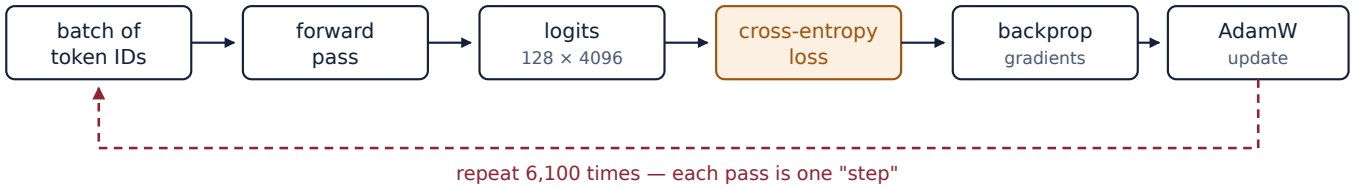
Where in ours: 4 heads × 32 dims = 128. The four projections (Q, K, V, output) are each 128 × 128, so attention costs $4 \times 128^2 =$ **65,536 parameters** per block.

We verified our implementation is correct. We wrote the equation out explicitly *and* wired up PyTorch's optimised kernel, then asserted the two agree — they match to `8.3e-07`. So the maths on this page is demonstrably what our code computes.

PART 8 · LEVEL 1 LEVEL 2

Training

The loop, in one picture



every 250 steps: run the same loop on VALIDATION data, but skip backprop and the update

every 500 steps: save a checkpoint to disk

The dashed red arrow is the only thing that changes the model. Everything else just measures.

Boxes are operations, solid arrows are data moving forward, the dashed red arrow is the weight update closing the loop.

Each stage explained

Forward pass

Push a batch of token sequences through the network and get logits out. For us: 64 sequences × 128 tokens → 64 × 128 × 4,096 numbers. No weights change; this is pure prediction.

★ Cross-entropy loss LEVEL 1

What: a single number measuring how wrong the predictions were.

How: take the probability the model gave to the *correct* token, take its natural logarithm, negate it, and average over all positions:

$$\text{Loss} = -\frac{1}{N}\sum \ln P(\text{correct token})$$

IF THE MODEL GAVE THE CORRECT TOKEN…	LOSS CONTRIBUTION
probability 1.00 (certain and right)	`−ln(1.00) = 0.00`
probability 0.50	`−ln(0.50) = 0.69`
probability 0.10	`−ln(0.10) = 2.30`
probability 1/4096 (pure guessing)	`−ln(1/4096) =` **`8.32`**

That last row is where our model started. Remember it — it is the whole random-init proof.

Backpropagation and gradients

Gradient: for each of the 1,331,968 weights, a number saying "if you increase this weight slightly, the loss changes by this much, in this direction".

Backpropagation: the algorithm that computes all 1,331,968 gradients efficiently by applying the chain rule backwards through the network. PyTorch's autograd does this for us — *we do not implement backprop by hand, and no one expects us to.*

★ AdamW — the optimizer

What it does: takes the gradients and decides how much to actually move each weight.

FEATURE	WHY IT HELPS
Momentum ($\beta_1 = 0.9$)	Averages recent gradients, so a single noisy batch doesn't throw the weights around.

Adaptive step size ($\beta_2 = 0.95$)	Weights with consistently large gradients get smaller steps; rarely-updated weights get larger ones.
Decoupled weight decay (0.1)	Gently shrinks weights towards zero — regularisation. The "W" in AdamW is that this is applied separately from the gradient, which is the correct way.

Our choice: we exclude LayerNorm parameters, biases and embeddings from weight decay — 786,432 weights decayed, 545,536 excluded. Decaying a LayerNorm gain fights the normalisation it exists to perform.

★ Learning rate

The single most important hyperparameter: the size of each step. Too small and training crawls; too large and it diverges. We used a **schedule**, not a fixed value:

```
warmup steps 0 → 305 (5%) lr rises linearly 0 → 3e-3 cosine steps 305 → 6,100 lr falls smoothly 3e-3 → 3e-4 (10% of peak)
```

Why warm up? At step 0 the weights are random and gradients are large and unreliable. Taking full-size steps immediately can destabilise the model before it has learned anything.

We chose the learning rate by experiment, not by guessing MEASURED

We ran three short 200-step probes from the identical random initialisation:

LEARNING RATE	VALIDATION LOSS AFTER 200 STEPS	VERDICT
5e-4	4.6537	stable but slow
1e-3	4.2026	stable
3e-3	3.8820	**selected** — best, and gradient norms fell as the rate rose

Saying "we ran a learning-rate sweep and chose empirically" is far stronger than "we used 3e-3".

Steps, epochs and batches

Batch	64 sequences processed together. Bigger batches give smoother gradients and better GPU use.
Step	One batch → one weight update. 64 × 128 = **8,192 tokens per step**.
Epoch	One full pass over the data. 6,100 steps × 8,192 = 49,971,200 tokens ≈ **one epoch-equivalent** of our 51.6M-token training set.

Be precise if asked: we sample random windows with replacement, so it is one epoch *equivalent in token count* — a few windows repeat, a few are never drawn. Saying "epoch-equivalent" rather than "epoch" is exactly the kind of care reviewers notice.

Validation

Every 250 steps we run the same forward pass on **held-out data the model never trains on**, and record the loss without updating anything. If training loss falls while validation loss rises, the model is memorising rather than learning — that is overfitting.

Ours did not overfit. Final validation loss **2.1407** finished *below* final training loss **2.194813**. With roughly one pass over the data, the model saw almost every token once — there was nothing to memorise. If asked "how do you know you didn't overfit?", that comparison is the answer.

PART 9 · LEVEL 1 — THE MOST IMPORTANT PART OF THE REVIEW

What "from scratch" means, and how to prove it

The distinction the panel is testing

APPROACH	WHAT HAPPENS	IS IT OURS?
Fine-tuning	`model = load("gpt2")` — you download weights someone else trained on billions of tokens, then adjust them slightly on your data.	No. The knowledge is theirs.
Transfer learning	Same, but you freeze part of the pretrained model.	No.
Using an API	You send text to someone else's model over the internet.	No. You have no model at all.
From scratch ✓	`model = OurTransformer(config)` — every weight is a fresh random number, and every bit of ability it later has came from our training loop on our data.	**Yes.**

The chain, start to finish

```
our architecture (code we wrote) ↓ RANDOM INITIALISATION every weight ~ N(0, 0.02), nothing loaded from anywhere ↓ our training data TinyStories, tokenized with our own tokenizer ↓ forward pass → loss → backpropagation → AdamW update × 6,100 ↓ OUR TRAINED CHECKPOINT ckpt_A_final.pt — 1,331,968 numbers we produced
```

What we DO use from PyTorch, and why that is fine

PyTorch gives us tensors, GPU kernels, automatic differentiation and the AdamW optimizer. Those are **tools**, like a compiler is a tool for writing a program. What PyTorch does *not* give us is the model: the architecture, the initialisation, the training loop and the resulting weights are all ours. If asked "isn't PyTorch doing the work?" — the answer is "PyTorch does the calculus; we designed the network and trained it."

The proof: cross-entropy at step 0 must equal ln(V)

Here is the argument in three lines. Learn it — it is the strongest thing you have.

1. A model that has learned nothing has no reason to prefer any token, so it spreads probability roughly evenly: each of the 4,096 tokens gets about 1/4096.
2. Cross-entropy is −ln P(correct token) = −ln(1/4096) = **ln(4096) = 8.3178**.
3. Our measured loss at step 0 was **8.3327** — a difference of +0.0150.

A model with pretrained weights cannot produce this number. It already knows that "the" is common and "zqx" is not, so its starting loss would be far below 8.32. One forward pass, one number, and the question is settled.

How I will prove this is my own from-scratch model

Ten pieces of evidence, in the order to present them.

#	EVIDENCE	WHAT TO SAY / SHOW
1	**Custom architecture**	Open `model/attention.py`, `block.py`, `transformer.py`. Every component written by us — the causal mask, the attention maths, the residuals.
2	**Custom tokenizer**	Our own BPE implementation. Vocabulary learned from our corpus, 3,966 merges. Not GPT-2's tokenizer, not anyone's.
3	**No pretrained loading call**	Run live: `pytest tests/test_no_pretrained.py -v`. It searches the whole repository for `from_pretrained`, `AutoModel`, `hf_hub_download` and fails the build if any appears. It also fails if `transformers` ever enters `requirements.txt`.
4	**Step-0 loss = ln(V)**	8.3327 vs ln(4096) = 8.3178. **The centrepiece.**
5	**Weight statistics at init**	Gaussian, mean ≈ 0, standard deviation 0.02003 — exactly the distribution we sample from. Show the histogram.
6	**Parameter count**	1,331,968, derived component by component. A unit test recomputes it from the config and asserts the value — the number on the slide is the number the code produces.
7	**Training logs**	`metrics.csv` opens at step 0 = 8.33214 and closes at step 6,100 = 2.194813. Every step in between is on disk.
8	**Loss curve**	Training and validation on one axis, with ln(4096) drawn as a reference line at the top.
9	**Checkpoint + reload**	16 MB file. Loaded in a fresh process, it reproduced validation loss 2.140691 with difference **0.000000**.
10	**Reproducibility**	Seed 1337 recorded in every run manifest, along with config, config hash, dataset counts, tokenizer fingerprint and hardware. Re-running gives the same numbers.

One more piece of evidence, if they push hard LEVEL 2

Because our LM head is tied to the token embedding, a common bug — targets not shifted by one — would make step-0 loss come out near **7.30** instead of 8.33. So the ln(V) check does double duty: it proves random initialisation *and* proves our training targets are correctly aligned. We have a unit test asserting exactly this.

PART 10 · LEVEL 1

Our model, parameter by parameter

If you can reproduce this table from memory, you can survive any architecture question.

Configuration and the reason for each choice

PARAMETER	VALUE	WHY THIS VALUE
Vocabulary size	4,096	Big enough for a simple corpus, small enough that the embedding table doesn't dominate a 1.3M model.
Embedding dim (d_model)	128	The width of the model's internal representation. Small but sufficient for TinyStories.
Number of layers	4	Enough depth to compose meaning across positions; shallow enough to train in minutes.
Attention heads	4	128 ÷ 4 = 32 dims per head. Heads must divide d_model exactly.
Head dimension	32	Sets the $\sqrt{d_k} = \sqrt{32}$ scaling in attention.
FFN dimension (d_ff)	512	4 × d_model — the conventional ratio.
Context length	128	How far back the model can see. TinyStories are short, so 128 tokens covers a whole story.
Positional encoding	learned	Learned absolute positions — the simplest correct choice. RoPE is planned for the bigger model.
Normalisation	pre-LN	Trains reliably without a hand-tuned warmup.
Activation	GELU	Standard, smooth, and keeps the parameter arithmetic simple.
LM head	tied	Shares the embedding matrix. Saves 524,288 parameters.
Total parameters	**1,331,968**	Derived below — never quoted as "about 1M" without the working.

The arithmetic — show this, don't just state the total

```
token embedding      4,096 × 128                        =   524,288
positional embedding   128 × 128                        =    16,384

per transformer block
  attention   W_q, W_k, W_v, W_o   4 × (128 × 128)      =    65,536
  2 layernorms                     2 × (2 × 128)        =       512
  feed-forward  128×512 + 512  +  512×128 + 128         =   131,712
  ------------------------------------------------------------------
  one block                                             =   197,760
  × 4 layers                                            =   791,040

final layernorm           2 × 128                       =       256
LM head                   tied to embedding             =         0
==================================================================
TOTAL                                                     1,331,968
```

The general formula, if you want one line to memorise:

`total = V·d + P + L·(4d² + 4d + 2·d·d_ff + d_ff + d) + 2d`

where V = vocab, d = d_model, L = layers, P = ctx·d for learned positions.

Where the parameters actually live

COMPONENT	PARAMETERS	SHARE
Token embedding (also the LM head)	524,288	39.4%
Feed-forward networks (4 blocks)	526,848	39.6%

Attention (4 blocks)	262,144	19.7%
Positional embedding	16,384	1.2%
LayerNorms (9 total)	2,304	0.2%

A good observation to volunteer: attention gets all the attention in papers, but the feed-forward networks actually hold slightly more of our parameters than attention does.

Scaling plan PROPOSED

CONFIG	PARAMS	D_MODEL	LAYERS	HEADS	CONTEXT	STATUS
A — MAX-1M	1,331,968	128	4	4	128	**trained**
B — MAX-14M	13,784,064	384	6	6	256	Review 2
C — MAX-29M	29,398,016	512	8	8	512	stretch goal

Having a scaling plan answers "why so small?" before it is asked: *we deliberately started at a size we could train, verify and debug in minutes, and the same code scales to 14M by changing a config file.*

PART 11 · LEVEL 2

Hardware

The vocabulary

TERM	WHAT IT MEANS FOR US
GPU	A processor with thousands of small cores. Neural networks are mostly matrix multiplication, which parallelises almost perfectly — this is why GPUs are 10–100× faster than CPUs here.
VRAM	Memory on the GPU. Must hold the weights, the gradients, the optimizer state, and the activations. Ours: **0.83 GB used of 15.4 GB available**.
CUDA	NVIDIA's platform for running general computation on their GPUs. PyTorch talks to CUDA for us.
Batch size	Sequences processed together — 64 for us. Larger = smoother gradients and better GPU use, but more VRAM.
Sequence length	Tokens per sequence — 128. Attention memory grows with the *square* of this, so it matters more than batch size.
Gradient accumulation	Run several small batches, add up their gradients, and update once. Simulates a big batch on a small GPU. We set it to 1 — we didn't need it.
Mixed precision	Store and compute in 16-bit instead of 32-bit floats. Roughly 2× faster and half the memory.

Our setup MEASURED

ITEM	VALUE
GPU	NVIDIA Tesla T4, 15.4 GB VRAM (free Google Colab)
Precision	fp16 + GradScaler
Batch × context	`64 × 128 = 8,192 tokens per step`
Peak VRAM used	`0.83 GB — about 5% of the card`
Throughput	`320,128 tokens/second`
Total training time	`2.6 minutes for 6,100 steps`

Why fp16 and not bf16 — a genuinely good detail to know

bf16 is the more forgiving 16-bit format, but it needs an NVIDIA Ampere GPU or newer. The T4 is **Turing**, one generation older, so bf16 is unavailable to us. fp16 has a much smaller numeric range, and small gradients underflow to zero — so a **GradScaler** is mandatory: it multiplies the loss by a large factor before backpropagation and divides it out before the update. Without it, fp16 training silently fails to learn.

Why a 1.33M model is practical, and a 1B model is not

MODEL SIZE	WEIGHTS + OPTIMIZER STATE	FEASIBLE ON A FREE T4?
1.33 M (ours)	≈ 21 MB	Trivially. Trains in minutes.
13.8 M (Review 2)	≈ 221 MB	Comfortably. Hours, not minutes.
124 M (GPT-2 small)	≈ 2 GB	Fits, but days of training for a decent result.
1 B	≈ 16 GB	No — the optimizer state alone fills the card.

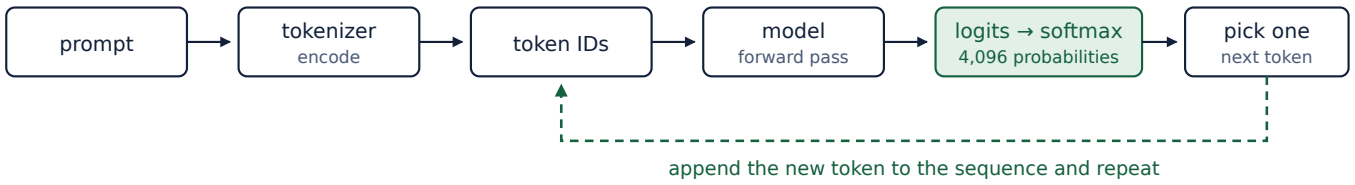
The memory arithmetic, if asked: AdamW stores the parameter, its gradient, and two momentum buffers — about **16 bytes per parameter** in fp32.

An honest detail worth mentioning

We estimated 15–45 minutes of training before running it. It actually took **2.6 minutes** — our estimate was pessimistic by roughly 6×, because we assumed a model this small would be bottlenecked by kernel launch overhead rather than arithmetic. We corrected the figure in our documentation. Volunteering a corrected estimate reads as scientific honesty, not error.

PART 12 · LEVEL 1

Inference — how the trained model produces text



stop when <|eos|> is produced, or when the token limit is reached

finally: decode the full ID sequence back to text with the tokenizer

Inference is the training forward pass, run repeatedly, with no loss and no weight update.

The green dashed arrow is what makes generation *autoregressive*: the model's own output becomes part of its next input.

How the next token is chosen

STRATEGY	WHAT IT DOES	EFFECT
Greedy (T = 0)	Always take the highest-probability token.	Deterministic. Safe but can loop.
Temperature sampling	Divide logits by T before softmax, then sample.	T < 1 sharpens (more conservative), T > 1 flattens (more varied).
Top-k	Keep only the k most likely tokens, then sample among them.	Prevents rare nonsense tokens. We use k = 50.

What our model actually produced MEASURED

T = 0.2 — "*The little girl was so happy that she had made a new friend. She was so happy that she could help her friend. She hugged her...*"

T = 0.7 — "*He decided not to be mad at the bear to be kind and not be grumpy. He said to himself, 'Come here, bear. You don't have to talk…*'"

T = 1.0 — "*, Max, he liked to play with his cat, and he played a game. One day, Hanny was playing tag and he saw a little puppy with a lost...*"

Grammatical sentences, correct pronoun agreement, coherent short narratives — from 1.33 million parameters trained for 2.6 minutes.

Say the limitation before the panel finds it

Look at the first sample: "*She was so happy that...*" appears twice. Our measured *immediate word* repetition rate is 0.11%, but that metric does not capture **phrase-level looping**, which is visible. Also: this is narrative English on a restricted vocabulary — it is **not** general language competence, and the model cannot answer questions. Naming your own model's weakness first is worth more than any metric that hides it.

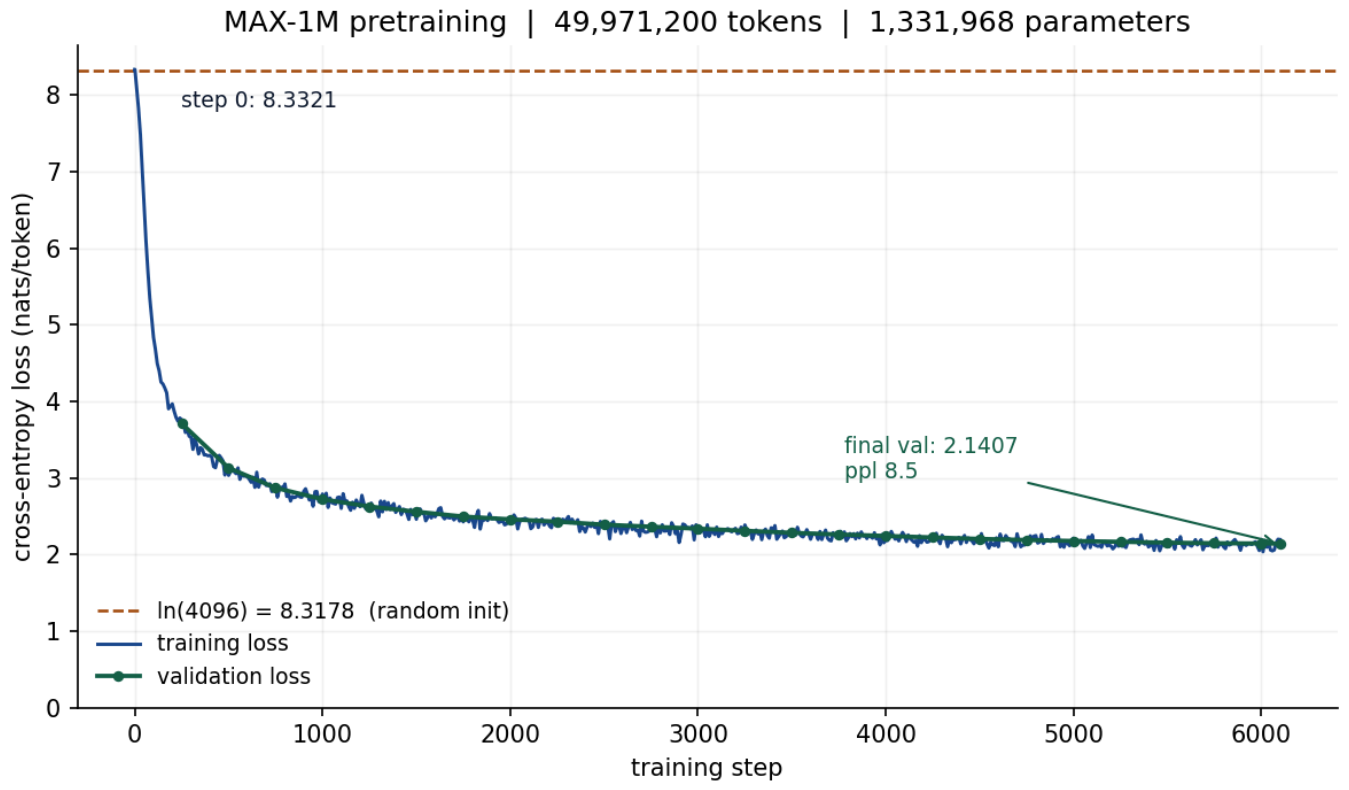
PART 13 · LEVEL 1

Evaluation

Every metric, what it means, and ours

METRIC	WHAT IT TELLS YOU	OURS MEASURED
Step-0 loss	Whether the model started from random weights. Must equal ln(vocab).	8.33214
Training loss	How well the model predicts data it is learning from. Falls as it learns.	2.194813
Validation loss	How well it predicts data it has never seen. The honest measure of learning.	2.1407
Perplexity	exp(loss). "On average the model is as uncertain as if choosing uniformly among this many tokens."	8.5053
Bits per token	loss ÷ ln 2. Comparable across different vocabulary sizes.	3.0884
Next-token accuracy@1	How often the single most likely token is the correct one.	51.10%
Reload difference	Whether the saved checkpoint reproduces the score in a fresh process.	0.000000
distinct-1 / distinct-2	Fraction of unique words / word-pairs in generated text. Low = repetitive.	0.3121 / 0.7016
Repetition rate	How often a word immediately repeats itself.	0.0011
Training time	Wall clock for the whole run.	2.6 min
Throughput	Tokens processed per second.	320,128/s
Peak VRAM	Maximum GPU memory used.	0.83 GB

The loss curve MEASURED



Reading it. The dashed line at the top is ln(4096) = 8.3178 — where a randomly initialised model must start, and where ours did. Blue is training loss, green is validation measured every 250 steps on data the model never trained on. The two curves sit on top of each other the whole way: no gap means no overfitting. The steep fall in the first ~300 steps is the model learning basic token frequencies; the long flat tail is it learning grammar and coherence.

★ Perplexity — the one you will definitely be asked about

Perplexity = e^{loss}

Plain-English meaning: "*at each position, the model is about as uncertain as if it were guessing uniformly between this many options.*"

SITUATION	LOSS	PERPLEXITY	INTERPRETATION
Random initialisation	8.3178	4,096	As uncertain as guessing among all 4,096 tokens
Our trained model	2.1407	8.51	As uncertain as choosing between ~8 or 9 options
Perfect prediction	0.0	1.0	Always certain and always correct (unattainable)

The sentence to say: "We went from a perplexity of 4,096 — complete uncertainty — down to 8.51. The model narrowed its choice from four thousand options to roughly eight. That is a **481× reduction in uncertainty**."

Why validation loss below training loss is a good sign

Normally validation loss is *higher* than training loss, because the model has seen the training data. Ours is lower (2.1407 vs 2.194813). Two reasons, and both are worth knowing:

- We trained for roughly **one epoch-equivalent**, so almost no example was seen twice — there was nothing to memorise.
- Training loss is averaged over the *whole run* including early high-loss steps, whereas validation is measured at the end. Comparing the final values is the fair comparison.

Either way, the conclusion is the same and it is the one that matters: **no overfitting**.

What we have NOT measured — say so plainly

QUANTITY	STATUS
Reasoning accuracy	**NOT YET MEASURED** — needs reasoning training (Review 2)
Multi-agent vs single-agent comparison	**NOT YET MEASURED** — needs Reviews 2 and 3
Confidence calibration (ECE, Brier)	**NOT YET MEASURED** — Review 4
Benchmark scores (e.g. GSM8K)	**NOT YET MEASURED** — and expected to be near zero at this scale
Human evaluation of explanations	**NOT YET MEASURED** — Review 4

"We haven't measured that yet" is a strong answer, not a weak one. It is the answer of someone who knows the difference between what they have done and what they plan to do. Guessing a number you did not measure is the fastest way to lose a panel's trust.

PART 14 · LEVEL 2

Sanity checks — how we know the pipeline is correct

A model can train smoothly and produce plausible numbers while being fundamentally broken. These checks catch that. We have **119 automated tests** that all pass.

#	CHECK	HOW WE VERIFY IT	RESULT
1	**Tokenizer round-trips**	Encode then decode 1,000 held-out documents; the output must equal the input exactly.	`1000/1000`
2	**Nothing silently vanishes**	A test asserts the pre-tokenizer's output pieces concatenate back to the original string.	`pass`
3	**Input/target shifted by one**	Assert `x[:, 1:] == y[:, :-1]`. If this fails, the model predicts its own input.	`pass`
4	**Parameter count correct**	A test recomputes the total from the config file and asserts it equals 1,331,968.	`pass`
5	**Weights are random**	Histogram plus statistics: mean ≈ 0, std 0.02003, LayerNorms at exactly 1.0.	`pass`
6	**Step-0 loss = ln(V)**	Tested at three different seeds; all within ±0.05 of 8.3178.	`pass`
7	**Overfit-one-batch**	Train on 8 fixed sequences only. A correct loop *must* drive the loss below 0.1. If it cannot memorise 1,024 tokens, more training will never help.	`pass`
8	**The model cannot see the future**	Change a later token and assert earlier positions' outputs are bit-identical. If this fails, every loss number is meaningless.	`pass`
9	**Every weight gets a gradient**	After one backward pass, check all 44 parameter tensors have non-zero gradients.	`44/44`
10	**Loss actually decreases**	The training log: 8.33214 at step 0 → 2.194813 at step 6,100.	`pass`
11	**Checkpoint saves and loads**	Reload in a fresh process; weights must be identical and validation loss must reproduce.	`Δ 0.000000`
12	**Wrong tokenizer is refused**	The checkpoint stores the tokenizer fingerprint `d6080bac0a9a`; the loader raises an error on mismatch instead of loading silently.	`pass`
13	**Inference works**	Generate from 5 prompts at 3 temperatures with recorded seeds; same seed gives same text.	`15 samples`
14	**No pretrained loading anywhere**	A test searches the entire repository and fails the build on any match.	`pass`

The two checks worth explaining in detail if asked

Overfit-one-batch — check #7

Give the model the same 8 sequences over and over with no regularisation. A 1.33M-parameter model must be able to memorise 1,024 tokens. If the loss does not collapse towards zero, something is broken: targets misaligned, gradients not flowing, learning rate dead, or the causal mask leaking. It takes under a minute and catches more bugs than any other single test.

Silent failure guards

Three of our tests exist because the corresponding bug produces *no error message*: targets off by one, gradient clipping applied before unscaling under fp16 (so it never actually fires), and loading a checkpoint against the wrong tokenizer (which yields fluent nonsense). Each looks like a normal run and produces wrong results.

PART 15 · LEVEL 1

Implementation roadmap — the exact order, and what "done" means

#	STEP	"DONE" MEANS	STATUS
1	Environment	PyTorch installed, GPU visible, repository structure created, all tests runnable.	✔
2	Dataset	Corpus downloaded, cleaned, deduplicated, split; document and character counts recorded in a manifest.	✔
3	Tokenizer	BPE trained on the train split; `tokenizer.json` saved with vocabulary and merge list.	✔
4	Tokenizer tests	Round-trip exact on held-out text; UNK rate measured; role tokens each encode to a single ID.	✔
5	Model	Attention, block and transformer implemented; forward pass returns the right shapes.	✔
6	Parameter verification	Counter prints 1,331,968 and matches the arithmetic component by component.	✔
7	Initialisation verification	Weight histogram is Gaussian; **step-0 loss equals ln(4096)**.	✔
8	Training-loop test	Overfit-one-batch drives loss below 0.1.	✔
9	LR probe	Three short runs at different learning rates; the best stable one selected.	✔
10	Full training	Run completes without NaNs; loss falls; `metrics.csv` written for every step.	✔
11	Validation	Validation loss measured on held-out data every 250 steps and at the end.	✔
12	Checkpoint	`ckpt_A_final.pt` exists with model, optimizer, scheduler, RNG states, config and tokenizer fingerprint.	✔
13	Inference	Checkpoint reloads in a fresh process, reproduces validation loss, and generates text.	✔
14	Evaluation	Loss, perplexity, accuracy, generation statistics and a loss curve produced.	✔
15	Documentation	Every run has a manifest with seed, config, config hash, dataset revision, tokenizer hash and hardware.	✔
16	Review presentation	Slides built, demo rehearsed, backup recording made.	in progress

Why the order matters — a good answer to "how did you approach this?"

Each step is a **gate**, not a task. We did not start the real training run until the overfit test passed, and we did not run the overfit test until step-0 loss matched ln(V). That ordering means that if something breaks, we know it broke in the step we just added — not somewhere in three thousand lines written blind.

PART 16 · LEVEL 1

What to show the panel

The demonstration order. Roughly 8 minutes.

#	SHOW THIS	SAY THIS
1	Problem statement slide	"A single reasoning process fails silently — one answer, no verification, no basis for trust."
2	Final architecture slide	"Five agents on one model, fused by a coordinator. That is the destination."
3	Review 1 scope slide	"Today is the foundation: our own language model, trained from scratch."
4	Dataset + tokenizer numbers	"226,654 stories, 4,096-token vocabulary we learned ourselves, 100% round-trip accuracy."
5	Config file + parameter counter	"1,331,968 parameters — and here is the arithmetic, component by component."
6	**Live:** `pytest tests/test_no_pretrained.py -v`	"This searches the whole repository for any pretrained-loading call. Zero matches."
7	**Step-0 loss beside ln(4096)**	"8.3327 against 8.3178. The model was uniform over the vocabulary — it knew nothing."
8	Weight histogram	"Gaussian, mean zero, standard deviation 0.02 — freshly sampled, not loaded."
9	Training log tail + loss curve	"8.33214 down to 2.194813 over 6,100 steps, in 2.6 minutes."
10	**Live:** load checkpoint in a fresh process, generate	"Reproduces validation loss exactly, then writes English. Note: it cannot answer questions — that is Review 2."
11	Evaluation table	"Validation 2.1407, perplexity 8.51, next-token accuracy 51.1%."
12	Limitations slide	State them before being asked. See Part 18, question 16.
13	Roadmap slide	"Review 2 reasoning, Review 3 multi-agent, Review 4 collaboration and explainability."

Three practical rules for the demo

- **Rehearse the `grep`/pytest command until it is instant.** Fumbling at a terminal in front of a panel undoes a good talk.
- **Record a clean run beforehand.** If the laptop or the network fails on the day, you play the recording and lose nothing.
- **Set expectations before generating.** Say "this is a 1.3M-parameter model, so expect simple sentences" *before* you press enter — not after someone looks unimpressed.

PART 17 · LEVEL 1 LEVEL 2

Viva questions — 42 of them

Written as a reviewer would ask them. **Short answer** = what to say first. **Better answer** = what to add if they follow up. **Testing** = why they asked.

A · Easy — the opening questions

1. What is your project about?

SHORT A system where several specialised AI agents work on the same question — one solves, one criticises, one verifies — and a coordinator combines them into an answer with an explanation of how it was reached.

BETTER Add: "Review 1 is the foundation — we built and trained the language model those agents will run on, from scratch."

Testing: can you explain your own project in plain language.

2. What exactly have you completed for Review 1?

SHORT A working 1,331,968-parameter decoder-only Transformer language model, built by us, initialised randomly and trained from scratch on ~50 million tokens. Validation loss 2.1407, perplexity 8.51, and it generates coherent English.

BETTER Walk the twelve-stage pipeline: dataset → preprocessing → tokenizer → encoding → model → random init → training → validation → checkpoint → reload → inference → evaluation.

Testing: is there a real artefact, or only slides.

3. Are you using ChatGPT or an existing LLM?

SHORT No. We implemented and trained our own decoder-only Transformer from random initialisation. No pretrained weights, no API model, at any stage.

BETTER "I can prove it in two ways: a test that fails the build if any pretrained-loading call appears in our code, and the step-0 loss of 8.3327, which is exactly ln(4096) — the score of a model that knows nothing."

Testing: the single most important claim of the review.

4. Why did you choose this project?

SHORT Because AI systems are increasingly trusted for reasoning, and a single-pass system gives no way to check whether its answer is sound. Multiple agents that criticise and verify each other address that directly.

Testing: motivation, and whether you understand the problem you chose.

5. What is a language model?

SHORT A function that, given a sequence of tokens, outputs a probability distribution over what the next token will be.

BETTER "Training uses only that one task — predict the next token — and it's self-supervised: the text is its own answer key, so no human labelling is needed."

Testing: fundamentals.

6. What dataset did you use and why?

SHORT TinyStories — short children's stories with a deliberately small vocabulary. 230,000 loaded, 226,654 after cleaning. License CDLA-Sharing-1.0.

BETTER "Eldan and Li showed in 2023 that models in the 1–30 million parameter range can produce fluent English on TinyStories. On Wikipedia at our scale you get gibberish. We chose the one corpus where a model our size is known to work."

Testing: was the choice reasoned or arbitrary.

7. How large is your model?

SHORT 1,331,968 parameters. Vocabulary 4,096, embedding dimension 128, 4 layers, 4 heads, feed-forward 512, context length 128.

BETTER Show the arithmetic: 524,288 embedding + 16,384 positional + 4 × 197,760 blocks + 256 final LayerNorm, with the LM head tied so it adds nothing.

Testing: do you know your own numbers, or did you write "~1M" and hope.

8. How long did training take?

SHORT 2.6 minutes on a free Tesla T4 — 6,100 steps, 49,971,200 tokens, 320,128 tokens per second, peak VRAM 0.83 GB.

Testing: whether the run really happened.

B · Medium — the follow-up questions

9. What is a token, and why not just use words?

SHORT A token is a chunk of text treated as one unit — usually a word or word-piece. Whole words would need a vocabulary of hundreds of thousands, larger than our entire model, and any unseen word becomes unknown.

BETTER "Characters would be the opposite problem — tiny vocabulary but very long sequences, wasting our 128-token context. Sub-word BPE is the compromise: 4,096 tokens, and rare words are built from pieces so nothing is truly unknown. Our measured UNK rate is 0.0000%."

Testing: do you understand the design trade-off, not just the definition.

10. Explain BPE.

SHORT Start with text as individual characters. Repeatedly find the most frequent adjacent pair and merge it into one new symbol, recording each merge. Stop at the target vocabulary size. Encoding new text means replaying those merges in order.

BETTER Give the worked example: with "low" appearing 5 times and "lower" twice, "l"+"o" is frequent, so it merges to "lo", then "lo"+"w" to "low". We learned 3,966 such merges.

Testing: can you explain an algorithm, not recite an acronym.

11. Why decoder-only? Why not the full encoder–decoder Transformer?

SHORT Encoder–decoder suits translation, where you read a complete input and then write a separate output. We only continue text left-to-right, so a single causal stack is the right shape — and it's what GPT-style models use.

Testing: architectural judgement.

12. What is self-attention?

SHORT A mechanism that lets each position in the sequence look at earlier positions and pull in the information relevant to it. In "the book … it was heavy", it lets "it" draw on "book".

BETTER Write the equation: $\mathbf{Attention(Q,K,V) = softmax(QK^s/\sqrt{d_k})V}$, and explain each piece.

Testing: the core technical concept of the whole architecture.

13. What are Q, K and V?

SHORT Query = what this position is looking for. Key = what each position offers for matching. Value = the information actually carried. All three are produced from the same input by three learned linear projections — which is why it is called *self*-attention.

BETTER The library analogy: query is your search term, keys are the labels on the spines, values are the contents you take away.

Testing: understanding versus memorisation.

14. Why divide by $\sqrt{d_k}$?

SHORT Without it, dot products grow large as dimension grows, pushing softmax into a nearly one-hot output where gradients vanish and learning stalls. Dividing by √32 keeps the scores in a workable range.

Testing: whether you understand the equation or just copied it.

15. What is causal masking and why do you need it?

SHORT During training the model sees the whole sequence at once. Causal masking sets all attention scores for future positions to $-\infty$ before the softmax, so their weight becomes exactly zero. Without it, position 5 could see position 6 and would be "predicting" a token it can already read.

BETTER "We test this rather than assume it: we change a later token and assert the earlier positions' outputs are bit-identical."

Testing: do you understand why the model doesn't cheat.

16. Why multiple attention heads?

SHORT Different heads can specialise — one on grammatical agreement, another on local word order, another on longer-range references. A single attention would have to average all those jobs together. We use 4 heads of 32 dimensions each.

Testing: architectural reasoning.

17. What is cross-entropy loss?

SHORT A measure of how wrong the predictions are: take the probability the model assigned to the correct token, take its natural log, negate it, average over all positions.

BETTER "If the model gives the right token probability 1.0, loss is 0. If it gives 1/4096 — pure guessing — loss is 8.32, which is exactly where our model started."

Testing: the training objective.

18. What is perplexity, and what does yours mean?

SHORT Perplexity is e raised to the loss. It means "the model is about as uncertain as if choosing uniformly among this many tokens". Ours is 8.51.

BETTER "We went from 4,096 at random initialisation to 8.51 — the model narrowed its effective choice from four thousand options to about eight. A 481-fold reduction in uncertainty."

Testing: can you interpret a metric, not just report it.

19. What is AdamW and why that optimizer?

SHORT An optimizer that combines momentum with per-parameter adaptive step sizes, plus decoupled weight decay. It is the standard choice for Transformers because it handles the very different gradient scales across embedding, attention and feed-forward parameters.

BETTER "We excluded LayerNorm parameters, biases and embeddings from weight decay — decaying a LayerNorm gain fights the normalisation it exists to perform. 786,432 weights decayed, 545,536 excluded."

Testing: did you configure it, or accept the default.

20. How did you choose the learning rate?

SHORT By experiment. We ran three 200-step probes from identical initialisation: 5e-4 gave validation 4.6537, 1e-3 gave 4.2026, 3e-3 gave 3.8820. We selected 3e-3.

BETTER "We also used a schedule rather than a fixed rate — 5% linear warmup then cosine decay to 10% of peak. Warmup matters because gradients at step 0 are large and unreliable."

Testing: empirical method versus copying a blog post.

21. What is the difference between training loss and validation loss?

SHORT Training loss is measured on data the model is learning from; validation loss on held-out data it never trains on. Validation is the honest measure of whether it learned anything general.

BETTER "If training loss falls while validation rises, that's overfitting. Ours finished at validation 2.1407 versus training 2.194813 — validation is *lower*, because we trained roughly one epoch-equivalent and there was nothing to memorise."

Testing: overfitting.

22. What is a batch, a step, and an epoch?

SHORT A batch is the group of sequences processed together — 64 for us. A step is one batch producing one weight update — 8,192 tokens. An epoch is a full pass over the data; our 6,100 steps is about one epoch-equivalent of our 51.6 million training tokens.

Testing: basic training vocabulary.

C · Technical — when they go deeper

23. Why do you need positional embeddings?

SHORT Attention has no inherent sense of order — it computes a weighted sum, which is the same regardless of arrangement. Without positional information "dog bites man" and "man bites dog" look identical to the model.

BETTER "We use learned absolute positions: a 128 × 128 table added to the token embedding, 16,384 parameters. For the larger model we plan RoPE, which costs no parameters and extrapolates better to longer sequences."

Testing: a classic and very likely question.

24. What are residual connections and why do they matter?

SHORT Instead of x = f(x), we compute x = x + f(x). Gradients can then flow backwards straight through the addition without passing through f, which is what makes deep networks trainable.

Testing: why deep networks work at all.

25. What does LayerNorm do, and why pre-LN?

SHORT It rescales a vector to mean 0 and variance 1, then applies a learned gain and bias, keeping activations numerically stable. Pre-LN means normalising *before* the sub-layer rather than after.

BETTER "Post-LN needs a carefully tuned warmup schedule or it diverges; pre-LN trains reliably out of the box. On a short project timeline, we could not afford a diverging run."

Testing: an implementation decision with a real justification.

26. What does the feed-forward network do that attention doesn't?

SHORT Attention mixes information *between* positions. The feed-forward network processes each position independently, adding non-linear computing capacity. You need both.

BETTER "Interestingly the FFNs hold slightly more of our parameters than attention does — 526,848 versus 262,144."

Testing: do you know what each part contributes.

27. What is weight tying?

SHORT The LM head shares the same matrix as the token embedding, used transposed. The embedding maps token → vector; the head maps vector → token score. They are inverse operations, so sharing is principled.

BETTER "It saves 524,288 parameters — 39% of our total. Without tying the model would be 1,856,256 parameters."

Testing: a detail only someone who built it would know.

28. How exactly do you initialise the weights?

SHORT Linear and embedding weights from a normal distribution with mean 0 and standard deviation 0.02; biases zero; LayerNorm gain 1 and bias 0.

BETTER "Residual projections are additionally scaled by $1/\sqrt{(2L)} = 1/\sqrt{8}$ — the GPT-2 recipe. Each block adds two contributions to the residual stream, so without that scaling the variance of the stream grows with depth. We measured 0.00707, exactly the target."

Testing: depth of implementation knowledge.

29. What is mixed precision, and why fp16 rather than bf16?

SHORT Computing in 16-bit instead of 32-bit floats — about twice as fast, half the memory. We use fp16 because the Tesla T4 is Turing architecture and bf16 requires Ampere or newer.

BETTER "fp16 has a narrow numeric range, so small gradients underflow to zero. A GradScaler is therefore mandatory: it multiplies the loss before backpropagation and divides out before the update. And clipping must come *after* unscaling, or you compare a scaled norm to an unscaled threshold and clipping never fires."

Testing: real hands-on experience. Very few students can answer the second half.

30. What does your checkpoint contain?

SHORT Model weights, optimizer state, scheduler state, random number generator states, step counter, tokens seen, the full config, its hash, and the tokenizer fingerprint. 16 MB.

BETTER "Weights alone would mean restarting from step zero after any interruption. And the tokenizer fingerprint is a guard — loading a checkpoint against a different tokenizer produces fluent nonsense with no error, so our loader refuses on mismatch."

Testing: engineering maturity.

31. How do you know your attention implementation is correct?

SHORT We implemented the equation explicitly and also wired up PyTorch's optimised kernel, then asserted the two produce the same output. They agree to 8.3×10^{-7}.

Testing: verification habits.

32. What is the overfit-one-batch test?

SHORT We train on 8 fixed sequences repeatedly. A correct training loop must drive the loss below 0.1 — a 1.33M-parameter model can certainly memorise 1,024 tokens. If it cannot, something is broken and more training will not fix it.

BETTER "It catches misaligned targets, dead gradients, a broken causal mask, and clipping that never fires — in under a minute, before we spend any real compute."

Testing: debugging methodology.

33. How much data would you need for a bigger model?

SHORT A common rule of thumb is roughly 20 tokens per parameter. For our planned 13.8M model that is about 276 million tokens — around 1.1 GB of text.

Testing: scaling awareness.

34. What is gradient clipping?

SHORT If the total gradient norm exceeds a threshold — 1.0 for us — we scale all gradients down proportionally. It prevents one unusual batch from taking a huge, destabilising step.

Testing: training stability.

D · Project-specific and trick questions

35. Your model can't answer questions. Isn't that a failure?

SHORT No — it's the expected result. A 1.33M-parameter model trained only on next-token prediction over children's stories learns to *continue text*, not to answer questions. Question-answering requires reasoning training, which is Review 2.

BETTER "Review 1's objective was to build and train a language model from scratch and demonstrate that it works as a language model. It does: validation loss 2.1407 and coherent English. Confusing that with question-answering would mean we had misunderstood our own milestone."

Testing: do you know the difference between a limitation and a failure. ***This is the most likely trap.***

36. Why not just fine-tune GPT-2? You'd get much better results.

SHORT We would get better text, but we would not have built a language model — and the project requirement is to build one.

BETTER "There's also a research advantage. Because we generated our own corpus and our own benchmark, we can state with certainty that no test item appeared in pretraining. With a downloaded model you can never rule out contamination — and our project's central claim is about comparing reasoning methods fairly."

Testing: do you understand why the constraint exists, or just that it was imposed.

37. Isn't PyTorch doing all the work for you?

SHORT PyTorch provides tensors, GPU kernels, automatic differentiation and the optimizer. It does not provide the model. The architecture, the initialisation, the training loop and the resulting weights are all ours.

BETTER "It's the same relationship as a compiler to a program. Using a compiler doesn't mean you didn't write the software."

Testing: composure under a slightly provocative question.

38. Your model is 1 million parameters. GPT-4 has trillions. What's the point?

SHORT We are not competing with GPT-4. Our research question is whether multiple collaborating agents reason better than a single agent — and that comparison is between our own methods, at matched cost, not against anyone else's model.

BETTER "A small model is actually an advantage for that question: we can run every experiment many times, with full control over the data, on free hardware. At GPT-4 scale we could run the study once, if at all."

Testing: scientific framing.

39. How do I know you actually trained this and didn't just download something?

SHORT The step-0 loss. It was 8.3327, and ln(4096) is 8.3178. A model with pretrained weights already knows which tokens are common and would start far below that.

BETTER Offer the whole evidence chain: the live repository search, the weight histogram, the complete training log from step 0, the loss curve, the seed and manifest for reproduction. "I can run any of these now."

Testing: the central claim, asked adversarially. Stay calm — you have more evidence than they expect.

40. What would you do differently if you started again?

SHORT Two things. Our training-time estimate was wrong by about 6× — we assumed the model would be bottlenecked by kernel launches rather than arithmetic. And we would add a phrase-level repetition metric, because our word-level one misses the looping that is actually visible in the samples.

Testing: self-awareness. Having a real answer here is worth a lot.

41. What are the risks to the rest of the project?

SHORT The main one: a model this small may not learn to reason well enough for the multi-agent comparison to show any difference. Our mitigation is a controlled synthetic reasoning benchmark sized to the model, rather than a public benchmark it would score zero on.

BETTER "A second risk is that the agents all produce the same output, which would make every aggregation method score identically. We plan to measure agent disagreement as a first-class metric from the first run, so we would catch that immediately."

Testing: have you thought past the current milestone.

42. What is your timeline for the rest?

SHORT Review 2: scale to a 13.8M model and add reasoning training. Review 3: the five agents and the coordination protocol. Review 4: collaborative neural reasoning, explainability, calibration and the full evaluation.

Testing: planning.

PART 18 · LEVEL 1

The 18 you must not get wrong

Memorise these answers close to word-for-word. If you know nothing else, know this page.

#	QUESTION	ANSWER TO MEMORISE
1	**What exactly have you built?**	A 1,331,968-parameter decoder-only Transformer language model, implemented by us, randomly initialised, and trained from scratch on ~50 million tokens of TinyStories using our own BPE tokenizer.
2	**Why from scratch?**	Because the goal is to build a language model, not to use one — and because owning the corpus means we can guarantee our later reasoning benchmark was never seen in pretraining.
3	**What is your architecture?**	Decoder-only Transformer: token + positional embeddings → 4 pre-LN blocks, each with 4-head causal self-attention and a 512-dim feed-forward, both residual → final LayerNorm → LM head tied to the embedding.
4	**How many parameters?**	1,331,968. Embedding 524,288 + positional 16,384 + four blocks at 197,760 + final LayerNorm 256. LM head tied, so it adds zero.
5	**What dataset?**	TinyStories. 230,000 loaded, 226,654 after cleaning and deduplication, 204 million characters, license CDLA-Sharing-1.0. Split 98/1/1.
6	**What tokenizer?**	Our own byte-pair encoding. 4,096 vocabulary, 3,966 merges learned from the training split only. 100% round-trip accuracy, 0.0000% unknown rate, 3.88 characters per token.
7	**What is BPE?**	Start with characters. Repeatedly merge the most frequent adjacent pair into one symbol, recording each merge. Stop at the target vocabulary size. Encoding replays the merges in order.
8	**What is attention?**	A mechanism letting each position look at earlier positions and pull in the information relevant to it. Attention(Q,K,V) = softmax($QK^T/\sqrt{d_k}$)V.
9	**What are Q, K, V?**	Query = what I'm looking for. Key = what I offer for matching. Value = the information I carry. All three come from the same input through three learned projections.
10	**What is causal masking?**	Setting attention scores for future positions to $-\infty$ before the softmax, so their weight is exactly zero. Without it the model could see the token it is meant to predict.
11	**What is cross-entropy?**	Negative log of the probability the model assigned to the correct token, averaged over positions. Probability 1 gives loss 0; probability 1/4096 gives loss 8.32.
12	**What is perplexity?**	e to the power of the loss. "As uncertain as guessing uniformly among this many tokens." We went from 4,096 to 8.51.
13	**How do you prove there are no pretrained weights?**	Step-0 loss was 8.3327 and ln(4096) = 8.3178 — the exact score of a model that knows nothing. A pretrained model starts far lower. Plus a test that fails the build if any pretrained-loading call appears anywhere in the code.
14	**What happens during training?**	Forward pass produces logits → cross-entropy measures the error → backpropagation computes a gradient for every weight → AdamW updates the weights. Repeat 6,100 times.
15	**What happens during inference?**	Prompt → tokenizer → token IDs → model → logits → softmax → pick a token → append it to the input → repeat until end-of-text or the limit. Then decode back to text.
16	**What are your limitations?**	The model is small, so it produces simple narrative English on a restricted vocabulary and cannot answer questions. Phrase-level repetition is still visible. Context is only 128 tokens. It has had no reasoning training yet.
17	**What are your results?**	Validation loss 2.1407, perplexity 8.5053, next-token accuracy 51.10%, trained in 2.6 minutes on a T4. The checkpoint reloads in a fresh process and reproduces the score exactly.

18	**What comes after Review 1?**	Review 2: reasoning training. Review 3: multi-agent architecture. Review 4: collaborative neural reasoning, explainability and evaluation.

Three things never to say

- **Never invent a number.** If you don't know it, say "I'd have to check the log." A wrong number that gets checked destroys everything else you said.
- **Never say "it works like ChatGPT".** It shares the architecture family; that is all. Say "it uses the same decoder-only Transformer architecture, at about 1/130,000 of the scale."
- **Never overclaim reasoning.** The model completes text. It does not reason, it does not answer questions, and saying otherwise invites a demo that contradicts you.

PART 19

Revision plans

The one-day plan — 9 hours with breaks

HOUR	TOPIC	WHAT TO DO — AND HOW YOU KNOW YOU'RE DONE
1	Project overview Parts 1–2	Read Parts 1 and 2. **Done when:** you can say what the project is, what Review 1 covers, and recite the twelve-stage pipeline without looking.
2	Dataset + preprocessing Part 3	Read Part 3. **Done when:** you can justify TinyStories in two sentences and explain why deduplication happens before splitting.
3	Tokenizer + BPE Part 4	Read Part 4 and *work the BPE example on paper*. **Done when:** you can run four merges on "low / lower / newest / widest" from memory.
4	LLM basics + Transformer Parts 5–6	Read Parts 5 and 6. **Done when:** you can draw the architecture diagram from memory and name what each box does.
5	Attention Part 7	Read Part 7. **Done when:** you can write the attention equation on paper and explain all four pieces, plus draw the causal mask triangle.
6	Training + loss + optimizer Part 8	Read Part 8. **Done when:** you can explain the loop in one breath and say why $-\ln(1/4096) = 8.32$.
7	From scratch + our model Parts 9–10	**The most important hour. Done when:** you can recite the ten pieces of proof and reproduce the parameter arithmetic.
8	Viva questions Parts 17–18	Read all 42 questions. Then cover the answers and try each one aloud. **Done when:** all 18 in Part 18 come out without hesitation.
9	Rehearsal	Present the slides out loud twice, timed. Run the demo commands twice. **Done when:** the demo takes under 8 minutes from a cold terminal.

If you only have half a day: hours 1, 3, 5, 7, 8. Skip Parts 11 and 14.

The 30-minute last-minute revision

MINUTES	DO EXACTLY THIS
0–5	Read Part 18 — the 18 answers. Out loud.
5–10	Memorise the six numbers below. Say them aloud until they're automatic.
10–15	Draw the architecture diagram and the twelve-stage pipeline on paper, from memory.
15–20	Write the attention equation and draw the causal-mask triangle.
20–25	Rehearse the three trick answers: questions 35, 36 and 39 in Part 17.
25–30	Say the opening line and the closing roadmap out loud, twice. Then stop.

The six numbers to have on the tip of your tongue

`1,331,968`	parameters
`8.3327`	step-0 loss — and ln(4096) = 8.3178
`2.1407`	final validation loss
`8.51`	perplexity (was 4,096 at initialisation)
`4,096`	vocabulary size
`2.6 min`	training time, 6,100 steps, Tesla T4

FINAL

If you remember only 20 things before walking into the review

#	
1	**We built a 1,331,968-parameter decoder-only Transformer ourselves and trained it from random initialisation.** That single sentence is the review.
2	**Step-0 loss was 8.3327. ln(4096) is 8.3178.** That is the proof no pretrained weights were used. Nothing else you say is as strong.
3	Final validation loss **2.1407**, perplexity **8.51**, next-token accuracy **51.10%**.
4	Dataset: **TinyStories**, 226,654 stories after cleaning, CDLA-Sharing-1.0 license. Chosen because models our size are *known* to produce fluent English on it.
5	Tokenizer: **our own BPE**, 4,096 vocabulary, 3,966 merges, fitted on the training split only.
6	BPE = repeatedly merge the most frequent adjacent pair of symbols, and record every merge in order.
7	Architecture: embeddings → **4 pre-LN blocks** (4-head causal attention + 512-dim FFN, both residual) → final LayerNorm → tied LM head.
8	**Attention(Q,K,V) = softmax($QK^s/\sqrt{d_k}$)V.** Q = what I want, K = what I offer, V = what I carry.
9	**Causal masking** sets future scores to −∞ so the model cannot see the token it must predict.
10	**Positional embeddings exist because attention has no sense of order** — otherwise "dog bites man" equals "man bites dog".
11	Training = forward pass → cross-entropy loss → backpropagation → AdamW update, repeated 6,100 times.
12	**Cross-entropy** = −ln(probability of the correct token). Perfect = 0, pure guessing over 4,096 tokens = 8.32.
13	**Perplexity = e^{loss}**. We went from 4,096 to 8.51 — a 481× reduction in uncertainty.
14	Learning rate **3e-3**, chosen by running three probes (5e-4 → 4.65, 1e-3 → 4.20, 3e-3 → 3.88), with 5% warmup and cosine decay.
15	Validation loss finished *below* training loss → **no overfitting**, because we trained about one epoch-equivalent.
16	The checkpoint **reloaded in a fresh process and reproduced the score with difference 0.000000**. It is a real, portable artefact.
17	**The model cannot answer questions — and that is expected, not a failure.** It was trained only to continue text. Reasoning is Review 2.
18	Limitations, said before being asked: small model, simple vocabulary, phrase-level repetition, 128-token context, no reasoning yet.
19	Roadmap: **R2** reasoning → **R3** multi-agent → **R4** collaborative reasoning + explainability.
20	**Never invent a number.** "I'd have to check the log" is a professional answer. A fabricated figure that gets checked ends the review.

The last thing to read before you walk in

You did not download a model and rename it. You wrote a tokenizer, wrote a Transformer, filled it with random numbers, and trained it until it learned to write English — and you have the logs, the tests and the checkpoint to prove every step. Walk in and describe what you did, plainly and in order. That is all a Review 1 panel wants.